Project Report: TrendPulse (Campus Placement Intelligence CLI)

1. Aim

Develop a library of specialist agents (e.g., Planner, Writer, Fundamentals Analyst, Search Agent) and coordinate their interactions. Use Tavily tool in the Search Agent to fetch live, accurate data, and Groq LLM to analyze and compile a custom placement strategy.

---

2. Theory

Agentic AI and Multi-Agent Coordination

Agentic AI represents a shift from passive language models to active, goal-driven systems that can plan, use external tools, reason, and self-correct. In a **Multi-Agent System (MAS)**, complex tasks are broken down and delegated to specialized agents. Each agent possesses unique domain knowledge, system prompts, and tools. Coordinating these agents requires establishing data pipelines, input-output validators, and orchestration logic.

Specialized Agents in TrendPulse

TrendPulse implements this specialized library pattern through modular software design, where each agent's logic is isolated:

1. **Search Agent (`TavilyService`)**:
- **Role:** Web Research & Information Gathering.
- **Tools:** Tavily Search API.
- **Logic:** Dynamically generates multiple optimized, domain-specific search queries based on the company name, role, and tech stack. It searches the live web, aggregates responses, extracts published dates, and deduplicates source URLs to present a clean, combined context.

2. **Planner Agent (`placement_prompt.py`)**:
- **Role:** Synthesis & Scheduling.
- **Logic:** Receives the user profile (target company, role, level, duration) and the Search Agent's output. It orchestrates how the study timeline is laid out (daily format for short durations, weekly format for longer durations to avoid cognitive overload and token limits).

3. **Writer Agent (`GroqService`)**:
- **Role:** Strategy Generation & Formatting.
- **Logic:** Communicates with the Groq API (using production models like `llama-3.3-70b-versatile`) to compile the final target summaries, prepare technical questions, write answer approaches, and build the day/week sprint tracks. It enforces strict JSON formatting. If the JSON is invalid, it triggers a self-correction retry loop.

4. **Fundamentals Analyst (`validators.py` & `analysis_models.py`)**:

- **Role:** Quality Control & Guardrails.
- **Logic:** Performs strict structural and data validation using Pydantic models. It validates the user inputs, ensures that the LLM output matches the required schema, identifies gaps in the user's tech stack, and outputs a realistic feasibility assessment.

---

3. Architecture

The interactions of the specialist agents are coordinated sequentially by the central app controller (`main.py`). Below is the visual representation of this multi-agent architecture:

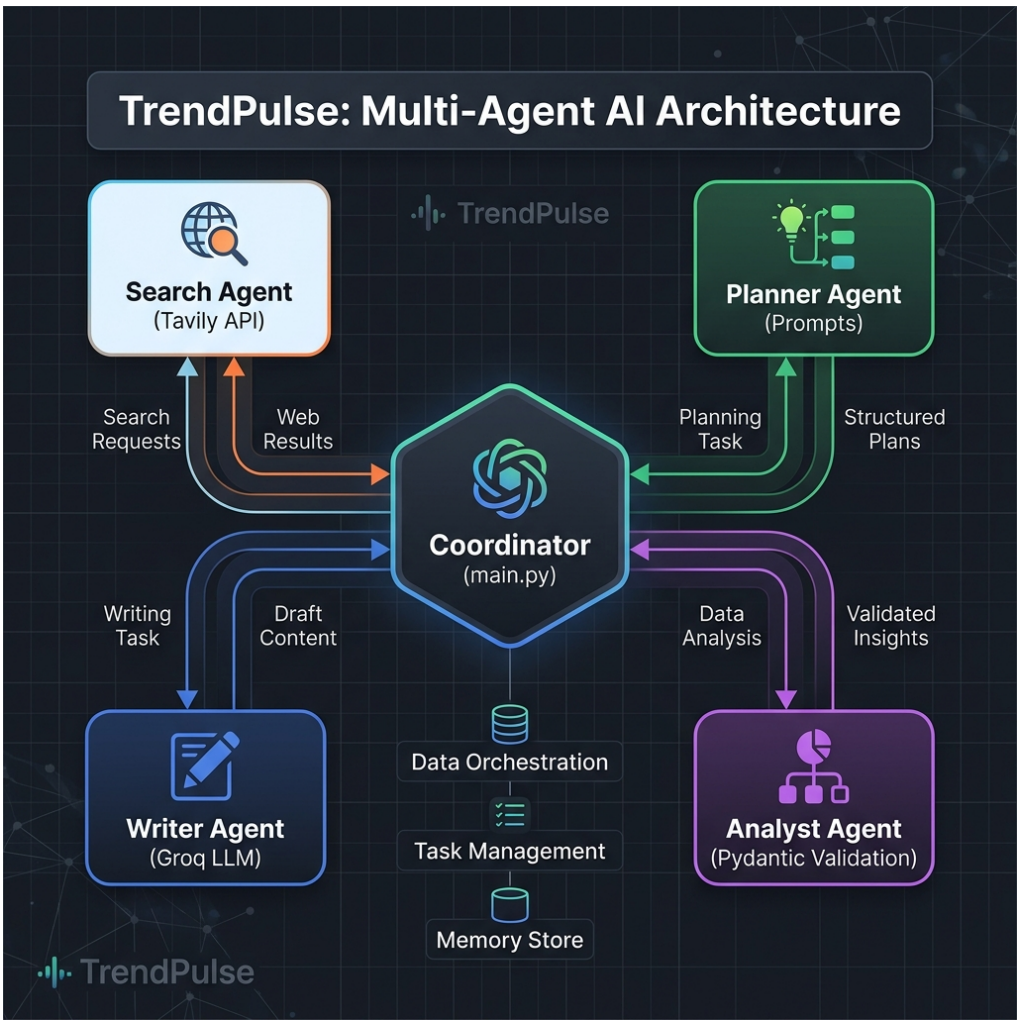


Figure: TrendPulse Multi-Agent Architecture

---

4. Folder Structure

```
trendpulse/
■
■■■ app/
■   ■■■ __init__.py
■   ■■■ main.py
■   ■■■ config.py
■   ■
■   ■■■ services/
■   ■   ■■■ __init__.py
■   ■   ■■■ tavily_service.py
■   ■   ■■■ groq_service.py
■   ■
■   ■■■ prompts/
■   ■   ■■■ __init__.py
■   ■   ■■■ placement_prompt.py
■   ■
■   ■■■ models/
■   ■   ■■■ __init__.py
■   ■   ■■■ analysis_models.py
■   ■
■   ■■■ utils/
■       ■■■ __init__.py
■       ■■■ terminal_ui.py
■       ■■■ validators.py
■
■■■ outputs/
■   ■■■ .gitkeep
■
■■■ .env
■■■ .env.example
■■■ .gitignore
■■■ requirements.txt
■■■ README.md
```

---

5. Code

requirements.txt

```
groq
tavily-python
rich
pydantic
python-dotenv
```

.env.example

```
TAVILY_API_KEY=your_tavily_api_key
GROQ_API_KEY=your_groq_api_key
GROQ_MODEL=llama-3.3-70b-versatile
TAVILY_MAX_RESULTS=5
REQUEST_TIMEOUT=30
```

app/config.py

```
import os
import sys
from dotenv import load_dotenv

def load_and_validate_env():
    """Load environment variables and validate required keys."""
    # Attempt to load from .env file
    load_dotenv()

    missing_keys = []

    tavily_key = os.getenv("TAVILY_API_KEY")
    groq_key = os.getenv("GROQ_API_KEY")
    groq_model = os.getenv("GROQ_MODEL")

    if not tavily_key:
        missing_keys.append("TAVILY_API_KEY")
    if not groq_key:
        missing_keys.append("GROQ_API_KEY")
    if not groq_model:
        missing_keys.append("GROQ_MODEL")

    if missing_keys:
        print("\n[!] Configuration Error")
        print("The following required environment variables are missing:")
        for key in missing_keys:
            print(f"  - {key}")
        print("\nPlease configure them in your .env file.")
        print("You can copy .env.example to .env and fill in the values.")
        sys.exit(1)

    # Optional parameters with defaults
    tavily_max_results = int(os.getenv("TAVILY_MAX_RESULTS", "5"))
    request_timeout = int(os.getenv("REQUEST_TIMEOUT", "30"))

    return {
        "tavily_api_key": tavily_key,
        "groq_api_key": groq_key,
        "groq_model": groq_model,
        "tavily_max_results": tavily_max_results,
        "request_timeout": request_timeout
    }
```

app/models/analysis_models.py

```
from typing import List, Literal, Optional
from pydantic import BaseModel, Field

# ------------------------------------------------------------
# Input Models
# ------------------------------------------------------------

class UserInput(BaseModel):
    company: str = Field(..., min_length=1)
    role: str = Field(..., min_length=1)
    tech_stack: str = Field(..., min_length=1)
    level: Literal["beginner", "intermediate", "advanced"]
    duration: Literal[7, 15, 30, 45, 60, 90, 120, 150, 180]

# ------------------------------------------------------------
# Internal Models
# ------------------------------------------------------------

class SearchResult(BaseModel):
    title: str
    url: str
    content: str
    published_date: Optional[str] = None

# ------------------------------------------------------------
# AI Output Models (Groq JSON schema)
# ------------------------------------------------------------

class LiveIntelItem(BaseModel):
    title: str
    description: str
    importance: Literal["High", "Medium", "Low"]
    source_urls: List[str]

class InterviewQuestion(BaseModel):
    question: str
    category: Literal["Coding", "Technical", "Aptitude", "HR", "System Design", "Other"]
    difficulty: Literal["Easy", "Medium", "Hard"]
    answer_approach: str
    source_urls: List[str]

class SprintStep(BaseModel):
    number: int
    unit: Literal["Day", "Week"]
    focus: str
    learn: List[str]
    practice: List[str]
    deliverable: str

class Source(BaseModel):
    title: str
    url: str
    published_date: Optional[str] = None

class PlacementReport(BaseModel):
    target_summary: str
    confidence_note: str
    feasibility_assessment: str
    tech_stack_gaps: List[str]
    live_intel: List[LiveIntelItem]
    recent_questions: List[InterviewQuestion]
```

```
sprint_plan: List[SprintStep]
final_strategy: List[str]
sources: List[Source]
```

app/prompts/placement_prompt.py

```
from app.models.analysis_models import UserInput, SearchResult
from typing import List

def generate_placement_prompt(user_input: UserInput, search_results: List[SearchResult]) -> str:
    """Generate the main prompt for Groq to analyze the placement data with the updated schema."""

    # Format the search results to be easily readable by the LLM
    search_context = ""
    for idx, res in enumerate(search_results, 1):
        search_context += f"Result {idx}:\n"
        search_context += f"Title: {res.title}\n"
        search_context += f"URL: {res.url}\n"
        search_context += f"Content: {res.content}\n"
        if res.published_date:
            search_context += f"Published: {res.published_date}\n"
        search_context += "-" * 40 + "\n"

    # Determine sprint steps format
    duration = user_input.duration
    if duration in [7, 15, 30]:
        unit = "Day"
        num_steps = duration
    else:
        unit = "Week"
        num_steps = duration // 7

    prompt = f"""
You are an expert AI campus placement advisor. Analyze the web research provided and generate a comprehensive

USER PROFILE:
- Target Company: {user_input.company}
- Target Role: {user_input.role}
- Tech Stack Provided: {user_input.tech_stack}
- Current Skill Level: {user_input.level}
- Preparation Duration: {user_input.duration} days

WEB RESEARCH RESULTS (Live Intel):
{search_context}

INSTRUCTIONS:
1. **Target Summary:** Synthesize the research to write a summary of the hiring processes, typical interview
2. **Confidence Note:** Include a disclaimer indicating whether the analysis is highly company-specific or ba
3. **Feasibility Assessment:** Write a realistic, constructive evaluation of whether the selected preparation
4. **Tech Stack Gaps:** Critically analyze the user's provided tech stack against what is typically expected
5. **Interview Questions:** Extract or construct 5 to 10 highly relevant recently reported or common question
6. **Sprint Plan:**
   - You MUST generate exactly {num_steps} entries in the `sprint_plan` list.
   - For this duration ({user_input.duration} days), the `unit` for each step must be "{unit}" (e.g. step 1,
   - Each step must have a clear Focus, Learn (array), Practice (array), and Deliverable (string).
7. Return your response strictly as a JSON object matching the schema below. DO NOT wrap it in markdown code

REQUIRED JSON SCHEMA:
{{
  "target_summary": "string",
  "confidence_note": "string",
  "feasibility_assessment": "string",
  "tech_stack_gaps": [
    "string"
  ],
  "live_intel": [
    {{
```

```
      "title": "string",
      "description": "string",
      "importance": "High | Medium | Low",
      "source_urls": ["string"]
    }}
  ],
  "recent_questions": [
    {{
      "question": "string",
      "category": "Coding | Technical | Aptitude | HR | System Design | Other",
      "difficulty": "Easy | Medium | Hard",
      "answer_approach": "string",
      "source_urls": ["string"]
    }}
  ],
  "sprint_plan": [
    {{
      "number": 1,
      "unit": "{unit}",
      "focus": "string",
      "learn": ["string"],
      "practice": ["string"],
      "deliverable": "string"
    }}
  ],
  "final_strategy": [
    "string"
  ],
  "sources": [
    {{
      "title": "string",
      "url": "string",
      "published_date": "string"
    }}
  ]
}}
"""
    return prompt
```

app/services/tavily_service.py

```
import json
import logging
from typing import List, Dict, Any
from tavily import TavilyClient

from app.models.analysis_models import SearchResult, UserInput

class TavilyService:
    def __init__(self, api_key: str, max_results: int = 5):
        self.client = TavilyClient(api_key=api_key)
        self.max_results = max_results

    def _generate_queries(self, user_input: UserInput) -> List[str]:
        """Generate search queries based on user target company, role, and stack."""
        company = user_input.company
        role = user_input.role
        stack = user_input.tech_stack
        return [
            f"{company} {role} interview experience 2026",
            f"{company} campus placement technical questions 2026",
            f"{company} {role} coding assessment skills recent",
            f"{company} {role} {stack} interview questions",
            f"{company} campus recruitment process {role}"
        ]

    def perform_research(self, user_input: UserInput) -> List[SearchResult]:
        """Perform multiple searches and combine unique results."""
        queries = self._generate_queries(user_input)
        all_results = []
        seen_urls = set()

        for query in queries:
            try:
                response = self.client.search(
                    query=query,
                    search_depth="basic",
                    max_results=self.max_results
                )

                for res in response.get("results", []):
                    url = res.get("url")
                    if url and url not in seen_urls:
                        seen_urls.add(url)
                        all_results.append(
                            SearchResult(
                                title=res.get("title", "Unknown Title"),
                                url=url,
                                content=res.get("content", ""),
                                published_date=res.get("published_date", "")
                            )
                        )
            except Exception as e:
                logging.debug(f"Tavily search failed for query '{query}': {e}")
                pass

        return all_results
```

app/services/groq_service.py

```
import json
import logging
from typing import List
from groq import Groq
from pydantic import ValidationError

from app.models.analysis_models import UserInput, SearchResult, PlacementReport
from app.prompts.placement_prompt import generate_placement_prompt

class GroqService:
    def __init__(self, api_key: str, model_name: str):
        self.client = Groq(api_key=api_key)
        self.model = model_name

    def _extract_json_from_markdown(self, text: str) -> str:
        """Safely extract JSON if wrapped in markdown code blocks."""
        text = text.strip()
        if text.startswith("```json"):
            text = text[7:]
        elif text.startswith("```"):
            text = text[3:]

        if text.endswith("```"):
            text = text[:-3]

        return text.strip()

    def generate_report(self, user_input: UserInput, search_results: List[SearchResult]) -> PlacementReport:
        """Send data to Groq and return a validated Pydantic model."""
        prompt = generate_placement_prompt(user_input, search_results)

        system_msg = {
            "role": "system",
            "content": "You are an expert AI career coach and placement strategist. You output ONLY valid JS
        }

        user_msg = {
            "role": "user",
            "content": prompt
        }

        messages = [system_msg, user_msg]

        try:
            response = self.client.chat.completions.create(
                model=self.model,
                messages=messages,
                temperature=0.3,
            )

            raw_text = response.choices[0].message.content
            cleaned_text = self._extract_json_from_markdown(raw_text)

            data = json.loads(cleaned_text)
            return PlacementReport(**data)

        except (json.JSONDecodeError, ValidationError) as e:
            logging.debug(f"First attempt failed: {e}. Retrying...")
            return self._retry_generation(messages, str(e), raw_text if 'raw_text' in locals() else "")

    def _retry_generation(self, original_messages: list, error_msg: str, previous_output: str) -> PlacementR
```

```
"""Retry generation asking Groq to fix the JSON."""
retry_messages = original_messages.copy()
if previous_output:
    retry_messages.append({
        "role": "assistant",
        "content": previous_output
    })
    retry_messages.append({
        "role": "user",
        "content": f"Your previous output caused a validation error:\n{error_msg}\n\nPlease provide 
    })
else:
    retry_messages.append({
        "role": "user",
        "content": "Ensure you return strictly valid JSON matching the requested schema."
    })

response = self.client.chat.completions.create(
    model=self.model,
    messages=retry_messages,
    temperature=0.1,
)

raw_text = response.choices[0].message.content
cleaned_text = self._extract_json_from_markdown(raw_text)

data = json.loads(cleaned_text)
return PlacementReport(**data)
```

app/utils/terminal_ui.py

```
from datetime import datetime
from rich.console import Console
from rich.panel import Panel
from rich.table import Table
from rich.prompt import Prompt, IntPrompt
from rich.markdown import Markdown
from app.models.analysis_models import PlacementReport

console = Console()

def display_welcome():
    welcome_text = "[bold cyan]TREND PULSE[/bold cyan]\n[italic]Live Placement Intelligence[/italic]"
    console.print(Panel(welcome_text, expand=False, border_style="cyan"))

def display_error(msg: str):
    console.print(Panel(f"[bold red]Error:[/bold red] {msg}", border_style="red"))

def prompt_user_input():
    company = Prompt.ask("[bold yellow]Target company or domain[/bold yellow]")
    role = Prompt.ask("[bold yellow]Target role[/bold yellow]")
    tech_stack = Prompt.ask("[bold yellow]Tech stack[/bold yellow]")

    level = Prompt.ask(
        "[bold yellow]Current level (beginner / intermediate / advanced)[/bold yellow]",
        choices=["beginner", "intermediate", "advanced"],
        show_default=False,
        show_choices=False
    )

    duration = IntPrompt.ask(
        "[bold yellow]Preparation duration in days (7 / 15 / 30 / 45 / 60 / 90 / 120 / 150 / 180)[/bold yello
        choices=["7", "15", "30", "45", "60", "90", "120", "150", "180"],
        show_default=False,
        show_choices=False
    )

    return {
        "company": company,
        "role": role,
        "tech_stack": tech_stack,
        "level": level,
        "duration": int(duration)
    }

def display_report(report: PlacementReport):
    # Target Summary
    console.print(Panel(f"{report.target_summary}\n\n[italic gray]{report.confidence_note}[/italic gray]", t

    # Feasibility Assessment
    console.print(Panel(report.feasibility_assessment, title="Feasibility Assessment", border_style="yellow"

    # Tech Stack Gaps
    if report.tech_stack_gaps:
        gaps_content = "\n".join([f"- {gap}" for gap in report.tech_stack_gaps])
        console.print(Panel(gaps_content, title="Tech Stack Gaps & Recommendations", border_style="red"))

    # Live Intel
    intel_table = Table(title="Live Intel", show_header=True, header_style="bold magenta")
    intel_table.add_column("Importance", style="dim", width=10)
    intel_table.add_column("Title", style="bold")
    intel_table.add_column("Description")
```

```
    for item in report.live_intel:
        color = "red" if item.importance == "High" else "yellow" if item.importance == "Medium" else "green"
        intel_table.add_row(f"[{color}]{item.importance}[/{color}]", item.title, item.description)
    console.print(intel_table)

    # Recently Reported Questions
    q_table = Table(title="Recently Reported Questions", show_header=True, header_style="bold green")
    q_table.add_column("Category")
    q_table.add_column("Difficulty")
    q_table.add_column("Question")
    q_table.add_column("Approach")

    for q in report.recent_questions:
        diff_color = "red" if q.difficulty == "Hard" else "yellow" if q.difficulty == "Medium" else "green"
        q_table.add_row(q.category, f"[{diff_color}]{q.difficulty}[/{diff_color}]", q.question, q.answer_app
    console.print(q_table)

    # Preparation Sprint
    console.print("\n[bold cyan]Preparation Sprint[/bold cyan]")
    for step in report.sprint_plan:
        content = f"[bold]Focus:[/bold] {step.focus}\n"
        content += f"[bold]Learn:[/bold] {', '.join(step.learn)}\n"
        content += f"[bold]Practice:[/bold] {', '.join(step.practice)}\n"
        content += f"[bold]Deliverable:[/bold] {step.deliverable}"
        console.print(Panel(content, title=f"{step.unit} {step.number}", border_style="cyan"))

    # Final Strategy
    strategy_content = "\n".join([f"- {s}" for s in report.final_strategy])
    console.print(Panel(strategy_content, title="Final Strategy", border_style="yellow"))

    # Sources
    console.print("\n[bold]Sources:[/bold]")
    for s in report.sources:
        console.print(f"- [link={s.url}]{s.title}[/link] ({s.published_date or 'No date'})")

def save_report_to_markdown(report: PlacementReport, user_input: dict) -> str:
    date_str = datetime.now().strftime("%Y-%m-%d")
    safe_company = "".join(c if c.isalnum() else "_" for c in user_input['company']).lower()
    safe_role = "".join(c if c.isalnum() else "_" for c in user_input['role']).lower()
    filename = f"outputs/{safe_company}_{safe_role}_{user_input['duration']}_day_report.md"

    with open(filename, "w", encoding="utf-8") as f:
        f.write(f"# TrendPulse Report\n")
        f.write(f"Generated date: {date_str}\n\n")
        f.write(f"**Target Company:** {user_input['company']}\n")
        f.write(f"**Target Role:** {user_input['role']}\n")
        f.write(f"**Tech stack:** {user_input['tech_stack']}\n")
        f.write(f"**Level:** {user_input['level']}\n")
        f.write(f"**Duration:** {user_input['duration']} days\n\n")

        f.write("## Target Summary\n")
        f.write(f"{report.target_summary}\n\n")
        f.write(f"_{report.confidence_note}_\n\n")

        f.write("## Feasibility Assessment\n")
        f.write(f"{report.feasibility_assessment}\n\n")

        f.write("## Tech Stack Gaps & Recommendations\n")
        for gap in report.tech_stack_gaps:
            f.write(f"- {gap}\n")
        f.write("\n")

        f.write("## Live Intel\n")
        for item in report.live_intel:
```

```
            f.write(f"- **[{item.importance}] {item.title}:** {item.description}\n")
        f.write("\n")

        f.write("## Recently Reported Questions\n")
        for q in report.recent_questions:
            f.write(f"### {q.question}\n")
            f.write(f"- Category: {q.category}\n")
            f.write(f"- Difficulty: {q.difficulty}\n")
            f.write(f"- Approach: {q.answer_approach}\n\n")

        f.write("## Preparation Sprint\n")
        for step in report.sprint_plan:
            f.write(f"### {step.unit} {step.number}: {step.focus}\n")
            f.write(f"- **Learn:** {', '.join(step.learn)}\n")
            f.write(f"- **Practice:** {', '.join(step.practice)}\n")
            f.write(f"- **Deliverable:** {step.deliverable}\n\n")

        f.write("## Final Strategy\n")
        for s in report.final_strategy:
            f.write(f"- {s}\n")
        f.write("\n")

        f.write("## Sources\n")
        for s in report.sources:
            f.write(f"- [{s.title}]({s.url})\n")
        f.write("\n")

        f.write("## Disclaimer\n")
        f.write("The report is generated from publicly available web results. Interview processes and questi

    return filename
```

app/utils/validators.py

```
from app.models.analysis_models import UserInput
from pydantic import ValidationError
from typing import Optional

def validate_user_input(data: dict) -> Optional[UserInput]:
    try:
        user_input = UserInput(**data)
        return user_input
    except ValidationError as e:
        return None
```

app/main.py

```
import sys
from rich.console import Console

from app.config import load_and_validate_env
from app.utils.terminal_ui import display_welcome, display_error, prompt_user_input, display_report, save_re
from app.utils.validators import validate_user_input
from app.services.tavily_service import TavilyService
from app.services.groq_service import GroqService

def main():
    console = Console()
    try:
        display_welcome()

        # Load environment
        config = load_and_validate_env()

        # Initialize services
        tavily_service = TavilyService(
            api_key=config['tavily_api_key'],
            max_results=config['tavily_max_results']
        )
        groq_service = GroqService(
            api_key=config['groq_api_key'],
            model_name=config['groq_model']
        )

        # Get and validate user input
        raw_input = prompt_user_input()
        user_input = validate_user_input(raw_input)

        if not user_input:
            display_error("Invalid input provided. Please ensure all fields are properly filled.")
            sys.exit(1)

        # Research Phase
        with console.status("[bold cyan]Gathering live intelligence...", spinner="dots"):
            search_results = tavily_service.perform_research(user_input)

        if not search_results:
            display_error("No relevant search results found. Please try broadening your target or tech stack
            sys.exit(1)

        # Analysis Phase
        with console.status("[bold magenta]Analyzing trends and generating strategy...", spinner="dots"):
            try:
                report = groq_service.generate_report(user_input, search_results)
            except Exception as e:
                display_error(f"Failed to generate analysis: {e}")
                sys.exit(1)

        # Display Results
        display_report(report)

        # Save Report
        try:
            filename = save_report_to_markdown(report, raw_input)
            console.print(f"\n[bold green]Success![/bold green] Report saved to [cyan]{filename}[/cyan]")
        except Exception as e:
            display_error(f"Failed to save Markdown report: {e}")

    except KeyboardInterrupt:
```

```
        console.print("\n[bold yellow]Process interrupted by user. Exiting...[/bold yellow]")
        sys.exit(0)
    except Exception as e:
        display_error(f"An unexpected error occurred: {e}")
        sys.exit(1)

if __name__ == "__main__":
    main()
```

---

6. Output Template

```
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ TREND PULSE                  ■
■ Live Placement Intelligence ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
Target company or domain: Google
Target role: Software Engineer
Tech stack: Python, SQL, Git, DSA
Current level (beginner / intermediate / advanced): beginner
Preparation duration in days (7 / 15 / 30 / 45 / 60 / 90 / 120 / 150 / 180): 15

■■■■■■■■■■■■■■■■■■■
```

```
■■■
```